T.C.

EGE UNIVERSITY

DEPARTMENT OF COMPUTER ENGINEERING

2024-2025 Spring Term

Programming Language Course Project 2 Report

INTERPRETER FOR THE PLUS++ PROGRAMMING LANGUAGE

Prepared by

Görkem Ege Akıncı, 05220000315

Yiğit Çakar, 05220000329

Zeynep Ünver, 05230000369

Course Instructor

Asst. Prof. Dr. Ahmet EGESOY

İZMİR, 2025

Table of Contents

Introduction **1**
Explanation of Source Code **4**
Libraries 5
Fundamental Structures 5
Big Int Functions 7
Parse Functions 12
Execute and Free Methods 25
Main Function with the Comment Operations Inside 28
How the Parser Works, How It Generates a Parse Tree, and How It Handles Errors **31**
How the Parser Works: 31
How the Parse Tree Is Generated: 32
How Errors Are Caught: 32
Test Cases and Results **32**
Sample Plus++ programs 32
Error Cases 34

1. Introduction

The purpose of our project is to develop an interpreter for the P++ language. It is a small programming language designed to handle big integers, perform basic operations such as increment, decrement and assignment, and display output. We implemented the interpreter in the C programming language.

When the program runs, it asks for the source file name in the terminal. The user types the filename without the .ppp extension, and the interpreter reads and executes the code inside that file.

During development, we made sure that the interpreter:

- Parses the source code line by line, handling multi-line comments and whitespace.
- Supports features including variable declarations, assignments, increment/decrement operations, write statements and loops.
- Generates a parse tree (AST) that represents the program structure for correct execution.
- Includes thorough error handling, reporting syntax and semantic errors clearly with line numbers, so users can easily identify and fix mistakes.

Our goal was to create an interpreter that is straightforward to use and strong in catching common programming errors in the code, making it easier for users to write correct programs.

2. Explanation of Source Code

2.1 Libraries

```c
// used libraries
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <ctype.h>
#include <limits.h>
```

In this project, we used standard C libraries to handle these operations:

- **stdio.h:** Input/output operations like printf() and scanf().
- **stdlib.h:** Memory allocation with malloc() and program termination with exit().
- **string.h:** String functions like copying, comparing and measuring length.
- **ctype.h:** Character checks such as isdigit().
- **limits.h:** Constants for data type limits like LLONG_MAX.

```c
// max limits
#define MAX_VAR_NAME    21
#define MAX_VARS        100
#define MAX_LINE_LEN    256
#define MAX_DIGITS      100
```

At the beginning of the code, we define some constants to set limits for certain things.

- **MAX_VAR_NAME** is 21, which means a variable name can be at most 20 characters long (plus the null character). This helps us avoid names that are too long.
- **MAX_VARS** is 100, so the program can store up to 100 different variables.
- **MAX_LINE_LEN** is set to 256 and it limits how long an input line can be. That way, we don't get problems with very long lines.
- **MAX_DIGITS** is 100 and it's used for the BigInt value. Since we store big numbers digit by digit, this puts a limit on how many digits a number can have.

2.2 Fundamental Structures

```c
// big integer struct
typedef struct {
    int digits[MAX_DIGITS + 2];
    int length;
    int negative;
} BigInt;
```

```c
// variable struct
typedef struct {
    char name[MAX_VAR_NAME];
    BigInt value;
    int initialized;
} Variable;
```

There are two main structs:

- **BigInt struct** stores very large integers as an array of digits. Least significant digit (LSD) is at index 0. The digits array size is MAX_DIGITS + 2 to prevent overflow while carrying during additions or subtraction. The length field shows how many digits the number has and the negative flag tells the sign: 1 means the number is negative, 0 means positive.
- **Variable struct** stores a variable's name, its value and an initialized flag. It tells if the variable has been assigned a value. It defines what information is stored for each variable in the symbol table. A symbol table is usually implemented as an array or a hash table.

```c
// enumeration for the parse tree
typedef enum {
    ND_VAR_DECL, ND_ASSIGN, ND_INC, ND_DEC,
    ND_WRITE, ND_LOOP, ND_BLOCK,
    ND_VAR_REF, ND_CONST, ND_STRING, ND_NEWLINE
} NodeType;
```

The **NodeType** enum defines different node types in the parse tree. Each node represents a part of the source code. For example:

- **ND_VAR_DECL:** variable declaration (int x;)
- **ND_ASSIGN:** assignment (x = 5;)
- **ND_INC:** increment (x++;)
- **ND_DEC:** decrement (x--;)
- **ND_WRITE:** output (write(x);)
- **ND_LOOP:** loop (while (x < 5) { ... })
- **ND_BLOCK:** code block ({ ... })
- **ND_VAR_REF:** using a variable (x)
- **ND_CONST:** constant value (10)
- **ND_STRING:** string literal ("hello")
- **ND_NEWLINE:** new line in the code

```c
// Node struct
typedef struct Node {
    NodeType type;
    struct Node *next;
    int var_index;
    BigInt const_val;
    char *string_val;
    struct Node *expr;
    struct Node *elements;
    struct Node *count;
    struct Node *body;
    struct Node *statements;
} Node;
```

The **Node** structure represents a single node in the parse tree. It supports various language constructs depending on the NodeType. Key fields include:

- **type:** the type of the node (e.g., ND_ASSIGN, ND_CONST).
- **next:** pointer to the next node at the same level (used for sequences like statement lists).
- **var_index:** index of a variable in the symbol table (used for declarations, references, and assignments).
- **const_val:** stores a constant value for ND_CONST nodes.
- **string_val:** stores string literals for ND_STRING nodes.
- **expr:** right-hand side expression for assignments, increments, or decrements.
- **elements:** used to store a list of expressions in ND_WRITE.
- **count:** the loop counter expression in ND_LOOP.
- **body:** the body of a loop (usually a block or single statement).
- **statements:** the list of child statements in a ND_BLOCK node.

```c
static Variable vars[MAX_VARS];
static int var_count = 0;
static char last_line_content[MAX_LINE_LEN] = "";
```

At the beginning, we have a list named **vars** that can hold up to MAX_VARS variables. This list keeps all the variables we create. We also use a number called **var_count** that starts at 0 and goes up by 1 every time we add a new variable. This helps us know how many variables we have so far. The variable **last_line_content** is a static character array used to store the content of the most recently processed line. It has a fixed maximum length defined by MAX_LINE_LEN.

```c
// function declarations
static Node *parse_statement(char **cursor,int *line_num,FILE *file);
static Node *parse_block(int *line_num,FILE *file,char *initial);
static Node *parse_int_value(char **cursor,int line_num);
static Node *parse_output_list(char *str,int line_num);
static void execute_node(Node *node);
static void free_node(Node *node);
```

These are static function declarations used for parsing the input, building the parse tree, executing statements and cleaning up memory.

```c
// error function
static void error(const char *msg, int line) {
    if (strcmp(msg, "Unrecognized statement.") == 0) {
        char trimmed[MAX_LINE_LEN];
        strcpy(trimmed, last_line_content);

        char *p = trimmed;
        while (isspace((unsigned char)*p)) ++p;

        char *e = p + strlen(p);
        while (e > p && isspace((unsigned char)*(e - 1))) --e;
        *e = '\0';

        if (strcmp(p, "}") == 0) return;
    }

    fprintf(stderr, "Error line %d: %s\n", line, msg);
    exit(1);
}
```

The **error** function displays an error message with the line number and terminates the program. It ignores lines that only contain a closing brace, treating them as harmless formatting issues. Before printing the error, it trims whitespace from the last read line.

2.3 Big Int Functions

```c
// BigInt functions
static int cmpMag(const BigInt *a, const BigInt *b) {
    if (a->length > b->length) return 1;
    if (a->length < b->length) return -1;
    for (int i = a->length - 1; i >= 0; --i) {
        if (a->digits[i] > b->digits[i]) return 1;
        if (a->digits[i] < b->digits[i]) return -1;
    }
    return 0;
}
```

The **cmpMag** function compares two BigInt numbers by their size. It first checks the number of digits, a longer number means a bigger value. When the lengths are equal, it compares the digits starting from the most significant one. It returns 1 or -1 depending on which is greater or 0 if they are exactly the same.

```c
static BigInt addMag(const BigInt *a, const BigInt *b) {
    BigInt r = { .negative = 0 };
    int carry = 0, max = a->length > b->length ? a->length : b->length, i;
    for (i = 0; i < max; ++i) {
        int sum = (i < a->length ? a->digits[i] : 0)
                + (i < b->length ? b->digits[i] : 0)
                + carry;
        r.digits[i] = sum % 10;
        carry = sum / 10;
    }
    if (carry) r.digits[i++] = carry;
    r.length = i ? i : 1;
    return r;
}
```

In **addMag**, we first find how many digits we need to add by checking the longer number. For each digit, it adds digits from both numbers (or 0 if shorter) plus any carry. The sum digit is saved and the carry updated. If there is still a carry at the end, it adds a new digit. Finally, it sets the total digits in the result. If the result has no digits, it sets it to zero.

```c
static BigInt subMagnitude(const BigInt *a, const BigInt *b) {
    // Assumes |a| >= |b|
    BigInt result;
    result.negative = 0;
    int borrow = 0;
    int i;
    for (i = 0; i < a->length; ++i) {
        int da = a->digits[i];
        int db = (i < b->length ? b->digits[i] : 0);
        int diff = da - db - borrow;
        if (diff < 0) {
            diff += 10;
            borrow = 1;
        } else {
            borrow = 0;
        }
        result.digits[i] = diff;
    }
    // Remove any leading zeros (at the high end of the number)
    int newLen = a->length;
    while (newLen > 1 && result.digits[newLen - 1] == 0) {
        newLen--;
    }
    result.length = newLen;
    if (result.length == 0) {
        result.length = 1;
        result.digits[0] = 0;
    }
    return result;
}
```

The **subMag** function subtracts one big number from another, assuming the first is larger or equal. We go through each digit, subtracting the second number's digit and any borrow from the first number's digit. When the result is negative, it adds 10 and borrow 1 for the next digit. After finishing all digits, we need to remove extra zeros at the end.

```
static BigInt bigAdd(BigInt x, BigInt y) {
    BigInt r;
    if (x.negative == y.negative) {
        r = addMag(&x, &y);
        r.negative = x.negative;
    } else {
        int c = cmpMag(&x, &y);
        if (c == 0) {
            r.length = 1;
            r.digits[0] = 0;
            r.negative = 0;
        } else if (c > 0) {
            r = subMag(&x, &y);
            r.negative = x.negative;
        } else {
            r = subMag(&y, &x);
            r.negative = y.negative;
        }
    }
    if (r.length == 1 && r.digits[0] == 0) r.negative = 0;
    return r;
}
```

The **bigAdd** function adds two big numbers with their signs which makes this method different from simple addition. When both have the same sign, it just adds their sizes and keeps that sign. If the signs are different, it compares their sizes and subtracts the smaller from the bigger. The result takes the sign of the bigger number. If the result is zero, the sign is set to positive so it's never negative zero.

```
static BigInt bigSub(BigInt a, BigInt b) {
    if (b.length == 1 && b.digits[0] == 0)
        b.negative = 0;
    else
        b.negative = !b.negative;
    return bigAdd(a, b);
}
```

The **bigSub** function subtracts one big number from another by changing the sign of the second number and then adding. This is a common method we learned in class and used here.

```c
// BigInt parse
static BigInt parseBigInt(const char *s, int line) {
    BigInt r;
    char buf[512];
    strncpy(buf, s, 511);
    buf[511] = '\0';

    char *p = buf;
    while (isspace((unsigned char)*p)) ++p;

    char *e = p + strlen(p);
    while (e > p && isspace((unsigned char)*(e - 1))) --e;
    *e = '\0';

    if (!*p) error("Invalid integer constant.", line);

    int neg = 0;
    if (*p == '-') {
        neg = 1;
        ++p;
        while (isspace((unsigned char)*p)) ++p;
    }

    if (*p == '+' || !isdigit((unsigned char)*p))
        error("Invalid integer constant.", line);
```

The **parseBigInt** function takes a string and turns it into a BigInt number. We copy the input into a buffer and trim spaces from both ends. If there's a minus sign at the beginning, we note that the number is negative and skip it along with any spaces after that. Next, we check if the remaining characters are all digits. If there's anything else in the rest of the string other than numbers or if the number is longer than 100 digits, we throw an error.

```c
    int neg = 0;
    if (*p == '-') {
        neg = 1;
        ++p;
        while (isspace((unsigned char)*p)) ++p;
    }
    if (*p == '+' || !isdigit((unsigned char)*p))
        error("Invalid integer constant.", line);

    int dcnt = 0;
    for (char *q = p; *q; ++q) {
        if (!isdigit((unsigned char)*q)) error("Invalid integer constant.", line);
        if (++dcnt > MAX_DIGITS) error("Integer constant too large.", line);
    }
    int nz = 0;
    while (nz < dcnt && p[nz] == '0') ++nz;
    if (nz == dcnt) {
        r.length = 1;
        r.digits[0] = 0;
        r.negative = 0;
        return r;
    }
    r.length = dcnt - nz;
    for (int i = 0; i < r.length; ++i)
        r.digits[i] = p[dcnt - 1 - i] - '0';

    while (r.length > 1 && r.digits[r.length - 1] == 0) --r.length;
    r.negative = neg;
    if (r.length == 1 && r.digits[0] == 0) r.negative = 0;
    return r;
}
```

When storing the number, it starts from the least significant digit (the rightmost one), so the digits go into the array in reverse order. We also skip leading zeros, since they don't change the value. This function also takes care of cases where the number is just zero, like "000". In that case, it stores it as a single 0 and makes sure the sign is positive because we don't want something like "-0".

```c
static void printBig(const BigInt *v) {
    if (v->negative) putchar('-');
    for (int i = v->length - 1; i >= 0; --i)
        printf("%d", v->digits[i]);
}
```

The **printBig** function prints a BigInt number. We print the digits from most to least significant. Since digits are stored in reverse, this makes the number appear correctly.

2.4 Parse Functions

```c
// parse int value
static Node* parse_int_value(char **cur, int line) {
    while (isspace((unsigned char)**cur)) ++*cur;
    if (!**cur) error("Invalid integer constant.", line);

    int neg = 0;
    if (**cur == '-') {
        neg = 1;
        ++*cur;
        while (isspace((unsigned char)**cur)) ++*cur;
    }

    char tok[256];
    int i = 0;
    while (**cur && (isalnum((unsigned char)**cur) || **cur == '_'))
        tok[i++] = **cur, ++*cur;
    tok[i] = '\0';

    if (!i) error("Invalid integer constant.", line);
```

The **parse_int_value** function reads an integer value from the input string. It skips spaces, handles optional minus signs and collects valid characters into a token. If the token is empty or invalid, it throws an error.

```c
    Node *n = calloc(1, sizeof(Node));
    if (isalpha((unsigned char)tok[0])) {
        int id = -1;
        for (int k = 0; k < var_count; ++k) {
            if (strcmp(vars[k].name, tok) == 0) {
                id = k;
                break;
            }
        }
        if (id == -1) error("Variable not declared.", line);
        n->type = ND_VAR_REF;
        n->var_index = id;
    } else {
        char nb[260];
        if (neg) {
            nb[0] = '-';
            strcpy(nb + 1, tok);
        } else {
            strcpy(nb, tok);
        }
        n->type = ND_CONST;
        n->const_val = parseBigInt(nb, line);
    }

    return n;
}
```

This code creates a new `Node` using `calloc` to ensure all fields are initialized to zero. It checks if the token is a variable name or a constant. If it's a variable, it looks it up in the symbol table and sets the node type to `ND_VAR_REF`. Otherwise, it treats it as a constant, handles the minus sign if needed and sets the node type to `ND_CONST` with its value.

```c
// write output list
static Node* parse_output_list(char *str, int line) {
    Node *head = NULL, *tail = NULL;
    char *p = str;

    while (1) {
        while (isspace((unsigned char)*p)) ++p;
        if (!*p) break;

        Node *el = NULL;

        if (*p == '"') {
            ++p;
            char buf[MAX_LINE_LEN];
            int bi = 0;

            while (*p && *p != '"') {
                if (bi < MAX_LINE_LEN - 1)
                    buf[bi++] = *p;
                ++p;
            }

            if (*p != '"')
                error("Invalid string literal.", line);

            buf[bi] = '\0';
            ++p;

            el = calloc(1, sizeof(Node));
            el->type = ND_STRING;
            el->string_val = strdup(buf);
        }
```

The **parse_output_list** function builds a linked list of nodes to be printed by a write statement. It skips whitespace and checks if the next item is a string. If so, it reads characters into a buffer until it finds the closing quote. Then, it creates a ND_STRING node using calloc and stores the string using strdup.

```c
        else if (strncmp(p, "newline", 7) == 0 && !isalnum((unsigned char)p[7])) {
            p += 7;
            el = calloc(1, sizeof(Node));
            el->type = ND_NEWLINE;
        }
        else {
            el = parse_int_value(&p, line);
        }

        if (!head)
            head = el;
        else
            tail->next = el;

        tail = el;

        while (isspace((unsigned char)*p)) ++p;

        if (strncmp(p, "and", 3) == 0 && !isalnum((unsigned char)p[3]))
            p += 3;
        else
            break;
    }

    return head;
}
```

If it detects the word newline, it creates an ND_NEWLINE node. Otherwise, it calls parse_int_value to handle constants or variable references. All nodes are created with calloc and linked together in order. The keyword and is used to separate multiple elements in the output list. The final result is the head of the linked list to be printed.

```c
/* ------------------------- Parse statement------------------------- */
static Node* parse_statement(char **cursor, int *line_num, FILE *file) {
    while (isspace((unsigned char)**cursor)) ++*cursor;
    if (!**cursor) return NULL;
    if (**cursor == '}') { ++*cursor; return NULL; }

    /* -------- Variable declaration -------- */
    if (strncmp(*cursor, "number", 6) == 0 && !isalnum((unsigned char)(*cursor)[6])) {
        *cursor += 6;
        while (isspace((unsigned char)**cursor)) ++*cursor;

        char *sc = strchr(*cursor, ';');
        if (!sc) error("Variable declaration must end with ';'.", *line_num);

        *sc = '\0';
        char *vn = *cursor;
        char *e = vn + strlen(vn);
        while (e > vn && isspace((unsigned char)*(e - 1))) --e;
        *e = '\0';

        if (!isalpha((unsigned char)vn[0]) || strlen(vn) > 20)
            error("Invalid variable name.", *line_num);
        for (char *c = vn; *c; ++c)
            if (!isalnum((unsigned char)*c) && *c != '_')
                error("Invalid variable name.", *line_num);
        for (int i = 0; i < var_count; ++i)
            if (strcmp(vars[i].name, vn) == 0)
                error("Variable already declared.", *line_num);
        if (var_count >= MAX_VARS)
            error("Too many variables.", *line_num);

        strncpy(vars[var_count].name, vn, MAX_VAR_NAME - 1);
```

This part of **parse_statement** handles variable declarations using the keyword `number`. It first checks for proper spacing and ensures the statement ends with a semicolon. Then it trims the variable name and validates it: the name must start with a letter, be at most 20 characters long, and only contain letters, digits, or underscores. It also checks for duplicate variable names and ensures the total number of variables does not exceed `MAX_VARS`. If any rule is violated, an error is reported with the corresponding line number.

```c
        vars[var_count].name[MAX_VAR_NAME - 1] = '\0';
        vars[var_count].value.length = 1;
        vars[var_count].value.digits[0] = 0;
        vars[var_count].value.negative = 0;
        vars[var_count].initialized = 1;

        int id = var_count++;
        *cursor = sc + 1;

        Node *n = calloc(1, sizeof(Node));
        n->type = ND_VAR_DECL;
        n->var_index = id;
        return n;
    }

    /* -------- repeat ... times loop -------- */
    if (strncmp(*cursor, "repeat", 6) == 0 && !isalnum((unsigned char)(*cursor)[6])) {
        *cursor += 6;
        while (isspace((unsigned char)**cursor)) ++*cursor;

        char *kw = strstr(*cursor, "times");
        if (!kw || (kw != *cursor && *(kw - 1) != ' '))
            error("Bad repeat syntax.", *line_num);

        *kw = '\0';
        char *cnt_str = *cursor;
        char *endcnt = kw;
        while (endcnt > cnt_str && isspace((unsigned char)*(endcnt - 1))) --endcnt;
        *endcnt = '\0';
```

This part of the parse_statement function handles two types of statements: variable declarations and repeat ... times loops. For variable declarations, it checks the variable name for length and character rules, ensures no duplicates, and enforces the MAX_VARS limit. If valid, it adds the variable to the symbol table with an initial value of zero and creates an ND_VAR_DECL node to represent it in the parse tree.

For loops, it detects the repeat keyword and checks that it is followed by a valid times keyword. The expression between repeat and times is trimmed and used as the loop count. If the syntax is incorrect, an error is reported. This ensures that loops follow the correct structure for further parsing and execution.

```c
        kw += 5;
        while (isspace((unsigned char)*kw)) ++kw;

        Node *cnt_node;
        int vid = -1;
        for (int i = 0; i < var_count; ++i)
            if (strcmp(vars[i].name, cnt_str) == 0) {
                vid = i;
                break;
            }
```

After locating the times keyword, the code moves the pointer past it and skips any spaces. Then, it tries to find if the loop count matches any declared variable name in the symbol table. If a match is found, it stores the variable's index for later use in the loop node. This step helps identify whether the loop count is a variable or a constant.

```c
        if (vid != -1) {
            cnt_node = calloc(1, sizeof(Node));
            cnt_node->type = ND_VAR_REF;
            cnt_node->var_index = vid;
        } else {
            cnt_node = calloc(1, sizeof(Node));
            cnt_node->type = ND_CONST;
            cnt_node->const_val = parseBigInt(cnt_str, *line_num);

            if (cnt_node->const_val.negative)
                error("Loop count negative.", *line_num);
            if (cnt_node->const_val.length > 19)
                error("Loop count too large.", *line_num);
            if (cnt_node->const_val.length == 19) {
                char buf[24];
                int j = 0;
                for (int i = 18; i >= 0; --i)
                    buf[j++] = '0' + cnt_node->const_val.digits[i];
                buf[j] = '\0';
                if (strcmp(buf, "9223372036854775807") > 0)
                    error("Loop count too large.", *line_num);
            }
        }

        Node *loop = calloc(1, sizeof(Node));
        loop->type = ND_LOOP;
        loop->count = cnt_node;
```

If the loop count matches a declared variable, a node of type ND_VAR_REF is created pointing to that variable. Otherwise, a constant node (ND_CONST) is created by parsing the count string as a big integer. The code checks that the loop count is not negative and is within a reasonable size limit to prevent errors. Finally, a new loop node (ND_LOOP) is created, storing the count node, which will be used later to execute the loop.

```c
        if (*kw == '\0') {  /* body starts next line */
            char nxt[MAX_LINE_LEN];
            if (!fgets(nxt, sizeof(nxt), file))
                error("Bad repeat syntax.", *line_num);
            (*line_num)++;
            nxt[strcspn(nxt, "\n")] = '\0';

            char *bl = nxt;
            while (isspace((unsigned char)*bl)) ++bl;
            if (*bl != '{') error("Bad repeat syntax.", *line_num);
            ++bl;
            while (isspace((unsigned char)*bl)) ++bl;
            char *end = bl + strlen(bl);
            while (end > bl && isspace((unsigned char)*(end - 1))) --end;
            *end = '\0';

            loop->body = parse_block(line_num, file, bl);
            *cursor += strlen(*cursor);
        } else if (*kw == '{') {  /* body starts same line */
            ++kw;
            while (isspace((unsigned char)*kw)) ++kw;
            char *end = kw + strlen(kw);
            while (end > kw && isspace((unsigned char)*(end - 1))) --end;
            *end = '\0';

            loop->body = parse_block(line_num, file, kw);
            *cursor += strlen(*cursor);
```

This part handles the loop body after the repeat ... times statement. If the opening brace { is not on the same line, it reads the next line from the file and checks for {. If found, it calls parse_block to parse the loop body starting from there. If the brace is on the same line, it skips whitespace and parses the block directly. This ensures the loop body is correctly read whether it starts on the same or the next line.

```c
        } else {  /* single-line loop */
            char *semi = strchr(kw, ';');
            if (!semi) error("Repeat single-line body must end with ';'.", *line_num);
            size_t len = semi - kw + 1;
            char body[MAX_LINE_LEN];
            strncpy(body, kw, len);
            body[len] = '\0';

            *cursor += (kw - *cursor) + len;
            char *bp = body;
            loop->body = parse_statement(&bp, line_num, file);
        }

        return loop;
    }
```

If the loop body is not enclosed in braces, it is treated as a single-line loop. The code looks for a semicolon to mark the end of the loop body, extracts that part as a string, and parses it as a single statement. This allows loops with simple one-line bodies without requiring braces. Finally, the function returns the constructed loop node.

```c
    /* -------- write -------- */
    if (strncmp(*cursor, "write", 5) == 0 && !isalnum((unsigned char)(*cursor)[5])) {
        *cursor += 5;
        while (isspace((unsigned char)**cursor)) ++*cursor;

        int n = strlen(*cursor);
        if (n == 0 || (*cursor)[n - 1] != ';')
            error("Write must end with ';'.", *line_num);

        (*cursor)[n - 1] = '\0';
        char *out = *cursor;
        char *e = out + strlen(out);
        while (e > out && isspace((unsigned char)*(e - 1))) --e;
        *e = '\0';

        *cursor += n;

        Node *w = calloc(1, sizeof(Node));
        w->type = ND_WRITE;
        w->elements = parse_output_list(out, *line_num);
        return w;
    }
```

This section parses the write statement. It first checks the keyword and skips any spaces. The statement must end with a semicolon (;); if not, an error is raised. The semicolon is removed, and trailing spaces are trimmed. Then, it creates a new ND_WRITE node and fills its elements by parsing the output list from the remaining string. This node will represent the output operation in the parse tree.

```c
    /* -------- assignment / inc / dec -------- */
    if (!isalpha((unsigned char)**cursor))
        error("Unrecognized statement.", *line_num);

    char lhs[MAX_VAR_NAME];
    int li = 0;
    while (**cursor && (isalnum((unsigned char)**cursor) || **cursor == '_')) {
        if (li < MAX_VAR_NAME - 1)
            lhs[li++] = **cursor;
        ++*cursor;
    }
    lhs[li] = '\0';

    while (isspace((unsigned char)**cursor)) ++*cursor;

    int op = -1;
    if ((*cursor)[0] == ':' && (*cursor)[1] == '=') op = 0;
    else if ((*cursor)[0] == '+' && (*cursor)[1] == '=') op = 1;
    else if ((*cursor)[0] == '-' && (*cursor)[1] == '=') op = 2;
    if (op == -1) error("Unrecognized statement.", *line_num);

    *cursor += 2;
    while (isspace((unsigned char)**cursor)) ++*cursor;

    int id = -1;
    for (int i = 0; i < var_count; ++i)
        if (strcmp(vars[i].name, lhs) == 0) {
            id = i;
            break;
        }
    if (id == -1) error("Variable not declared.", *line_num);
```

This part parses assignment and increment/decrement statements. It first reads the variable name on the left side, then checks the operator (:=, +=, or -=). If the operator is invalid, it reports an error. Next, it looks up the variable name in the symbol table to ensure it is declared. If the variable is not found, an error is raised. This prepares the parse tree node for further expression parsing.

```c
    Node *rhs = parse_int_value(cursor, *line_num);
    while (isspace((unsigned char)**cursor)) ++*cursor;
    if (**cursor != ';') error("Missing ';'.", *line_num);
    ++*cursor;

    Node *n = calloc(1, sizeof(Node));
    n->type = (op == 0 ? ND_ASSIGN : (op == 1 ? ND_INC : ND_DEC));
    n->var_index = id;
    n->expr = rhs;
    vars[id].initialized = 1;

    return n;
}
```

After parsing the right-hand side value, it checks for a terminating semicolon. Then, it creates a new node representing the operation: assignment, increment, or decrement. The node links to the variable

and the value expression. Finally, it marks the variable as initialized and returns the node for further processing.

```c
static Node* parse_block(int *line_num, FILE *file, char *initial) {
    Node *blk = calloc(1, sizeof(Node));
    blk->type = ND_BLOCK;
    Node *tail = NULL;

    if (initial && *initial) {
        char *p = initial;
        while (1) {
            while (isspace((unsigned char)*p)) ++p;
            if (!*p) break;

            if (*p == '}') {
                ++p;
                while (isspace((unsigned char)*p)) ++p;
                if (*p) strncpy(last_line_content, p, MAX_LINE_LEN - 1);
                return blk;
            }

            Node *st = parse_statement(&p, line_num, file);
            if (!st) break;
            if (!blk->statements) blk->statements = st;
            else tail->next = st;
            tail = st;
            while (tail->next) tail = tail->next;
            while (isspace((unsigned char)*p)) ++p;
            if (!*p) break;
            if (*p == '}') {
                ++p;
                while (isspace((unsigned char)*p)) ++p;
                if (*p) strncpy(last_line_content, p, MAX_LINE_LEN - 1);
                return blk;
            }
```

The first part of the parse_block function handles inline blocks that are passed as a string through the initial parameter. This typically occurs when a code block starts on the same line as a control statement (e.g., repeat ... times { ... }). The parser skips whitespace and checks whether the block ends immediately with a closing brace (}). If so, it stores any remaining characters for future parsing. Otherwise, it attempts to parse statements one by one using parse_statement, and appends them to the block's linked list structure. After each statement, it continues to look for a closing brace. This part is crucial for handling compact, single-line code blocks.

```c
    char buf[MAX_LINE_LEN];
    while (fgets(buf, sizeof(buf), file)) {
        (*line_num)++;
        buf[strcspn(buf, "\n")] = '\0';

        char *ln = buf;
        while (isspace((unsigned char)*ln)) ++ln;

        char *e = ln + strlen(ln);
        while (e > ln && isspace((unsigned char)*(e - 1))) --e;
        *e = '\0';

        if (!*ln) continue;

        if (*ln == '}') {
            ++ln;
            while (isspace((unsigned char)*ln)) ++ln;
            if (*ln) strncpy(last_line_content, ln, MAX_LINE_LEN - 1);
            return blk;
        }

        char *p = ln;
        while (1) {
            while (isspace((unsigned char)*p)) ++p;
            if (!*p) break;

            if (*p == '}') {
                ++p;
                while (isspace((unsigned char)*p)) ++p;
                if (*p) strncpy(last_line_content, p, MAX_LINE_LEN - 1);
                return blk;
```

The second part comes into play if the block is not fully contained in the initial string. It reads lines from the input file one at a time, incrementing the line counter and trimming whitespace. If a line contains only a closing brace (}), the block ends and any remaining characters are saved. Blank lines are ignored. This mechanism allows the parser to correctly process multi-line blocks that span across several lines in the source file, ensuring that the block boundaries are clearly identified.

```c
            Node *st = parse_statement(&p, line_num, file);
            if (!st) break;

            if (!blk->statements) blk->statements = st;
            else tail->next = st;

            tail = st;
            while (tail->next) tail = tail->next;

            if (st->type == ND_LOOP) {
                while (*p && *p != '\0') ++p;
                break; // consume rest if loop
            }

            while (isspace((unsigned char)*p)) ++p;
            if (!*p) break;

            if (*p == '}') {
                ++p;
                while (isspace((unsigned char)*p)) ++p;
                if (*p) strncpy(last_line_content, p, MAX_LINE_LEN - 1);
                return blk;
            }
        }
    }

    error("Missing closing brace for repeat block.", *line_num);
    return NULL;
}
```

The third part is responsible for parsing each individual statement within the block and managing loop-specific behavior. It uses parse_statement to interpret each piece of the line. If a loop statement is detected, the parser consumes the rest of the line to avoid reprocessing the loop body. The parsed statement is added to the block, and the parser checks again for a closing brace. This part is essential for constructing the abstract syntax tree (AST) properly and for maintaining accurate block structure, especially in the presence of nested or complex statements. If no closing brace is found by the end of the file, an error is raised to signal an incomplete block.

2.5 Execute and Free Methods

```c
static void execute_node(Node *n) {
    if (!n) return;
    switch (n->type) {
        case ND_BLOCK:
            for (Node *s = n->statements; s; s = s->next)
                execute_node(s);
            break;
        case ND_VAR_DECL:
            break;
        case ND_ASSIGN: {
            BigInt v = (n->expr->type == ND_VAR_REF) ? vars[n->expr->var_index].value : n->expr->const_val;
            vars[n->var_index].value = v;
            break;
        }
        case ND_INC: {
            BigInt v = (n->expr->type == ND_VAR_REF) ? vars[n->expr->var_index].value : n->expr->const_val;
            vars[n->var_index].value = bigAdd(vars[n->var_index].value, v);
            break;
        }
        case ND_DEC: {
            BigInt v = (n->expr->type == ND_VAR_REF) ? vars[n->expr->var_index].value : n->expr->const_val;
            vars[n->var_index].value = bigSub(vars[n->var_index].value, v);
            break;
        }
```

The first part of the **execute_node** function is responsible for executing different types of AST (Abstract Syntax Tree) nodes. It starts by checking for a null node and returns immediately if so. For a ND_BLOCK type, which represents a block of statements, it iteratively calls execute_node on each statement in the block. For assignment-related nodes (ND_ASSIGN, ND_INC, and ND_DEC), it either retrieves the value from a variable or uses a constant value and performs the appropriate operation. Assignments directly overwrite the variable's value, while increment and decrement nodes use helper functions bigAdd and bigSub to compute the result and update the variable accordingly. This part ensures that the memory model and variable values are updated based on the logic defined in the abstract syntax tree.

```c
        case ND_WRITE: {
            for (Node *e = n->elements; e; e = e->next) {
                if (e->type == ND_STRING)
                    printf("%s", e->string_val);
                else if (e->type == ND_NEWLINE)
                    putchar('\n');
                else if (e->type == ND_VAR_REF)
                    printBig(&vars[e->var_index].value);
                else if (e->type == ND_CONST)
                    printBig(&e->const_val);
            }
            break;
        }
        case ND_LOOP: {
            BigInt cnt = (n->count->type == ND_VAR_REF) ? vars[n->count->var_index].value : n->count->const_val;
            long long c = 0;
            for (int i = cnt.length - 1; i >= 0; --i)
                c = c * 10 + cnt.digits[i];

            int lvid = (n->count->type == ND_VAR_REF) ? n->count->var_index : -1;

            for (long long cur = c; cur > 0; --cur) {
                if (lvid != -1) {
                    long long t = cur;
                    BigInt b = {.length = 0, .negative = 0};
                    if (t == 0) {
                        b.length = 1;
                        b.digits[0] = 0;
                    }
```

The second part handles the ND_WRITE and ND_LOOP node types. For ND_WRITE, it iterates through a list of output elements and prints them to the console. This can include strings, newlines, variable values, or constant numbers. It uses different methods to print each type correctly, ensuring formatted output. For ND_LOOP, the code first evaluates the loop count (either from a constant or variable), converts it to a long long, and then performs a standard for-loop. If the loop count is a variable, its index is stored so it can be updated during each iteration. This section is crucial for output and control flow operations, managing both the interaction with the user (via output) and repeated execution (via loops).

```
                    while (t) {
                        b.digits[b.length++] = t % 10;
                        t /= 10;
                    }
                    vars[lvid].value = b;
                }

                if (n->body->type == ND_BLOCK) {
                    for (Node *s = n->body->statements; s; s = s->next)
                        execute_node(s);
                } else {
                    execute_node(n->body);
                }
            }

            if (lvid != -1) {
                vars[lvid].value.length = 1;
                vars[lvid].value.digits[0] = 0;
                vars[lvid].value.negative = 0;
            }

            break;
        }

        default:
            break;
    }
```

The third part completes the loop execution logic. If the loop count is associated with a variable, that variable is updated each iteration to reflect the current loop value, stored as a BigInt. The loop body may be a block of statements or a single statement, and both cases are handled accordingly. After the loop finishes, if a loop variable was used, it is reset to zero. This reset ensures a clean state after loop execution. The logic in this part is key for ensuring loop control variables behave predictably and for supporting dynamic loop bounds when using variables.

```c
static void free_node(Node *n) {
    if (!n) return;

    if (n->type == ND_BLOCK) {
        Node *s = n->statements, *nx;
        while (s) {
            nx = s->next;
            free_node(s);
            s = nx;
        }
    }
    else if (n->type == ND_WRITE) {
        Node *e = n->elements, *nx;
        while (e) {
            nx = e->next;
            free_node(e);
            e = nx;
        }
    }
    else if (n->type == ND_LOOP) {
        free_node(n->count);
        free_node(n->body);
    }
    else if (n->type == ND_ASSIGN || n->type == ND_INC || n->type == ND_DEC) {
        free_node(n->expr);
    }
    else if (n->type == ND_STRING) {
        free(n->string_val);
    }

    free(n);
}
```

The **free_node** function safely frees memory used by AST nodes. It handles different node types like blocks, loops, and write statements by recursively freeing their subnodes (e.g., statements, elements, count, body). For assignments and expressions, it frees the expr field, and for string nodes, it also frees the allocated string. Finally, it frees the node itself. This ensures all dynamically allocated memory is properly released, preventing memory leaks.

2.6 Main Function with the Comment Operations Inside

```c
int main(int argc, char **argv) {
    char fname[128];
    if (argc >= 2) {
        strncpy(fname, argv[1], 127);
        fname[127] = '\0';
    }
    else {
        printf("Source file (without .ppp): ");
        if (scanf("%127s", fname) != 1) return 1;
    }
    strcat(fname, ".ppp");
    FILE *fp = fopen(fname, "r");
    if (!fp) {
        perror(fname);
        return 1;
    }

    Node *program = calloc(1, sizeof(Node));
    program->type = ND_BLOCK;
    Node *tail = NULL;

    char line[MAX_LINE_LEN];
    int ln = 0, in_comment = 0;
```

The program gets the file name from the command line arguments. If no name is given, it asks the user to type the name. Then it adds ".ppp" to the file name and opens the file. If it cannot open the file, it prints an error and stops. It also creates an empty program block and prepares variables for reading the file.

```c
    while (fgets(line, sizeof(line), fp)) {
        ln++;
        line[strcspn(line, "\n")] = '\0';

        /* strip *...* comments (may span lines) */
        char res[MAX_LINE_LEN] = "";
        char *src = line, *dst = res;

        while (*src) {
            if (!in_comment) {
                if (*src == '*') {
                    in_comment = 1;
                    ++src;
                    continue;
                }
                else {
                    *dst++ = *src++;
                }
            }
            else {
                char *end = strchr(src, '*');
                if (end) {
                    src = end + 1;
                    in_comment = 0;
                }
                else {
                    src += strlen(src);
                }
            }
        }
```

The program reads the file line by line. It removes comments written between asterisks (*) that can span multiple lines. If a comment is not finished on one line, it keeps reading the next lines until the comment ends. The cleaned line is stored for further use.

```c
        *dst = '\0';
        strcpy(line, res);
        if (in_comment) continue;

        char *lnptr = line;
        while (isspace((unsigned char)*lnptr)) ++lnptr;

        char *end = lnptr + strlen(lnptr);
        while (end > lnptr && isspace((unsigned char)*(end - 1))) --end;
        *end = '\0';

        if (!*lnptr) continue;
        if (*lnptr == '}') continue;

        strncpy(last_line_content, lnptr, MAX_LINE_LEN - 1);

        char *p = lnptr;
```

After cleaning the line, it removes extra spaces from the start and end. If the line is empty or only a closing brace '}', it skips it. It saves the last non-empty line for error tracking.

```c
        while (1) {
            while (isspace((unsigned char)*p)) ++p;
            if (!*p) break;

            Node *st = parse_statement(&p, &ln, fp);
            if (!st) break;

            if (!program->statements)
                program->statements = st;
            else
                tail->next = st;

            tail = st;
            while (tail->next) tail = tail->next;

            if (st->type == ND_LOOP) {
                while (*p && *p != '\0') ++p;
                break; /* skip rest of line */
            }

            while (isspace((unsigned char)*p)) ++p;
            if (!*p) break;
        }
    }

    fclose(fp);

    execute_node(program);
    free_node(program);
    return 0;
```

The program processes the line to get statements one by one. It adds each parsed statement to the program's list. If it finds a loop statement, it skips the rest of the line to avoid parsing mistakes. After reading all lines, it closes the file.

3. How the Parser Works, How It Generates a Parse Tree, and How It Handles Errors

3.1 How the Parser Works:

The parser reads the .ppp file line by line. Before parsing, it removes multi-line comments marked by *...*, trims whitespace and skips empty lines or lines containing only closing braces. The main parsing happens in the parse_statement function, which analyzes each line and identifies statements based on the language syntax:

- **Variable Declarations:** Detects the number keyword, validates the variable name and syntax, stores variables, and creates ND_VAR_DECL nodes.

- **Loops:** Recognizes repeat ... times, parses the loop count (constant or variable), validates it, and parses the loop body (block or single statement), creating ND_LOOP nodes.
- **Write Statements:** Parses write statements with strings, newlines, or variables, and builds ND_WRITE nodes.
- **Assignments and Arithmetic Operations:** Detects assignment and update statements (:=, +=, -=), verifies variable existence, and parses right-hand expressions.

3.2 How the Parse Tree Is Generated:

The parser builds a hierarchical tree with nodes representing different statement types (ND_VAR_DECL, ND_ASSIGN, ND_LOOP...). Nodes are linked sequentially via next pointers. Each node contains data specific to its type. At the root is an ND_BLOCK node containing all top-level statements. This tree models the program's syntax and enables correct sequential execution by the interpreter.

3.3 How Errors Are Caught:

Error handling is centralized in a function that prints an error message with the line number and immediately terminates the program. This prevents running invalid code. The parser checks for various errors, including:

- **Syntax Errors:** Missing semicolons, braces, or unexpected keywords.
- **Variable Errors:** Invalid names, redeclarations, or use of undeclared variables.
- **Value Errors:** Invalid or oversized integers, negative loop counts.
- **String Errors:** Unclosed string literals.
- **Unrecognized Statements:** Any unknown constructs (except lines with only closing braces).

By carefully parsing line by line and validating syntax and semantics, the parser ensures only well-formed PPP programs proceed to execution, providing useful error feedback to the user.

4. Test Cases and Results

4.1 Sample Plus++ programs

Test 1: Variable Declarations and Assignments

```
number a;
number b;
number result;

a := 12345;
b := -67890;
result := a;
result += b;

write "Result is:" and result and newline;
```

```
Source file (without .ppp): test_variables
Result is:-55545
```

Test 2: Incrementing and Decrementing with Big Numbers

```
number a;
number b;
number c;

a := 1234567890123456789012345678901234567890123456789012345678901234567890123456789012345678901234567890;
b := 9876543210987654321098765432109876543210987654321098765432109876543210987654321098765432109876543210;

c := a;
c += b;

write c and newline;
```

```
Source file (without .ppp): test_big_numbers
11111111101111111110111111111101111111110111111111101111111110111111111101111111110111111111101111111100
```

Test 3: The write Statement, and Keyword, and newline Usage

```
write "Hello," and "world!" and newline;
write "This is line 1." and newline and "This is line 2." and newline;
write "1 + 1 = " and 2 and newline;
```

```
Source file (without .ppp): test_write
Hello,world!
This is line 1.
This is line 2.
1 + 1 = 2
```

Test 4: Using a Variable as a Counter in repeat...times Loop

```
number i;
i := 4;

repeat i times
{
    write "Iteration:" and i and newline;
}

write "Final value of i:" and i and newline;
```

```
Source file (without .ppp): test_repeat_var_counter
Iteration:4
Iteration:3
Iteration:2
Iteration:1
Final value of i:0
```

Test 5: Nested Loops and Code Blocks

```
number outer;
number inner;

outer := 2;

repeat outer times
{
    write "Outer loop value:" and outer and newline;
    inner := 3;
    repeat inner times
    {
        write "Inner loop value:" and inner and newline;
    }
}
```

```
Source file (without .ppp): test_nested_loops
Outer loop value:2
Inner loop value:3
Inner loop value:2
Inner loop value:1
Outer loop value:1
Inner loop value:3
Inner loop value:2
Inner loop value:1
```

Test 6: Fixed-Value repeat...times Loop

```
repeat 10 times write "x"; *writes 10 x on the screen*
```

```
Source file (without .ppp): test_repeat_constant
xxxxxxxxxx
```

4.2 Error Cases

Error 1: Undeclared Variable Usage

```
undeclared.ppp
x := 5;
write x;
```

```
Error line 1: Variable not declared.
```

Error 2: Syntax Error – Missing Semicolon

```
missing_semicolon.ppp
number x
x := 5;
```

```
Source file (without .ppp): missing_semicolon
Error line 1: Variable declaration must end with ';'
```

Error 3: Invalid Integer Format – Leading Plus Sign

```
invalid_number_format_plus.ppp
number x;
x := +5;
```

```
Source file (without .ppp): invalid_number_format_plus
Error line 2: Invalid integer constant.
```

Error 4: Invalid Statement Keyword

```
invalid_keyword.ppp
num x;
```

```
Source file (without .ppp): invalid_keyword
Error line 1: Unrecognized statement.
```

Error 5: Repeat with Negative Loop Count

```
repeat_negative.ppp
repeat -2 times write "hello" and newline;
```

```
Source file (without .ppp): repeat_negative
Error line 1: Loop count negative.
```

Error 6: Variable Already Declared

```
duplicate_variable.ppp
number x;
number x;
```

```
Source file (without .ppp): duplicate_variable
Error line 2: Variable already declared.
```

Error 7: Too Long Integer (>100 digits)

too_large_integer.ppp

```
number x;
x := 1234567890123456789012345678901234567890123456789012345678901234567890123456789012345678901;
```

```
Source file (without .ppp): too_large_integer
Error line 2: Integer constant too large (more than 100 digits).
```